

Core CS & Software Engineering Roadmap

A deep-fundamentals track for Shahruk — running alongside the React + Next.js course — aimed at becoming a strong engineer in the AI era and performing well in technical interviews.

How to use this roadmap

This is a parallel track, not a replacement for the React/Next.js course. The course teaches you the framework layer; this roadmap builds the foundation underneath it — the things interviewers probe and the things that separate someone who can follow a tutorial from someone who can reason about a system. Treat it as ongoing background study: a fixed daily block (even 1.5–2 hours) is better than sporadic long sessions. Everything here is sequenced so earlier topics unlock later ones.

Suggested daily split while both tracks are active: reserve your main study block for the React/Next.js course and building projects (including TaskForge AI), and use a separate fixed block each day for DSA practice plus one fundamentals topic below. Interview prep (Phase 7) should run continuously in the background from Phase 2 onward, not just at the end.

PHASE 1: Programming Fundamentals, Deep (~3-4 weeks)

1.1 JavaScript, beyond the basics

- Execution context, scope chain, closures — not just the definition, but predicting output of tricky code
- The 'this' keyword: rules, and how arrow functions differ
- Prototypes and prototypal inheritance (before jumping to classes)
- Event loop, call stack, microtasks vs macrotasks, callback queue — trace real execution order
- Promises, async/await internals; error handling patterns
- Higher-order functions, currying, closures used for memoization/private state
- Memory: primitive vs reference types, garbage collection basics

1.2 TypeScript fundamentals

- Static typing basics: types, interfaces, generics, union/intersection types
- Why TypeScript matters for full-stack roles and how it changes how you write JS

1.3 Programming paradigms

- Imperative vs declarative, procedural vs object-oriented vs functional — recognize each in code you read
- OOP pillars: encapsulation, inheritance, polymorphism, abstraction — with real small examples
- Functional concepts: pure functions, immutability, side effects

Why this phase first: almost every DSA and system-design conversation assumes fluency here. Interviewers frequently probe closures, 'this', and async behavior directly.

PHASE 2: Data Structures & Algorithms (DSA) (~8-10 weeks, ongoing after)

2.1 Complexity analysis

- Big-O, Big-Theta, Big-Omega — time and space complexity
- Analyzing loops, recursion, and nested structures for complexity

2.2 Core data structures

- Arrays and strings (two pointers, sliding window)
- Linked lists (singly, doubly, fast/slow pointers)
- Stacks and queues (incl. monotonic stack/queue patterns)
- Hash tables/maps and sets — collision handling, when to reach for them
- Trees: binary trees, binary search trees, traversals (DFS/BFS, in/pre/post-order)
- Heaps/priority queues
- Graphs: adjacency list/matrix, BFS, DFS, topological sort
- Tries (useful for autocomplete/prefix problems)

2.3 Core algorithm techniques

- Sorting (merge, quick, heap sort) and searching (binary search variants)
- Recursion and backtracking
- Dynamic programming — start with 1D (Fibonacci, climbing stairs), progress to 2D (grid, knapsack, LCS)

- Greedy algorithms
- Graph algorithms: shortest path (Dijkstra), union-find/disjoint set

2.4 Practice strategy

- Learn a pattern, then drill 8-12 problems on that exact pattern before moving on (pattern-first, not random)
- Track problems in a spreadsheet: pattern, difficulty, whether solved independently, revisit date
- Revisit missed problems after 1 week and 1 month (spaced repetition)
- Once patterns are solid, mix random problems to simulate interview conditions

This is the single highest-leverage phase for technical interviews. Keep a light daily DSA habit running continuously even after moving to later phases.

PHASE 3: Computer Systems: OS, Memory, Networking (~3-4 weeks)

3.1 How computers work (foundation)

- CPU, RAM, storage, how a program actually runs — just enough to reason about performance
- Compilation vs interpretation vs JIT (where does JS/Node fit?)

3.2 Operating systems

- Processes vs threads, concurrency vs parallelism
- Memory management: stack vs heap, virtual memory basics
- CPU scheduling basics, deadlocks (conceptual understanding, common interview topic)
- How Node.js's single-threaded event loop relates to OS-level concurrency

3.3 Networking fundamentals

- OSI model basics (just enough to place HTTP, TCP, IP)
- TCP vs UDP, three-way handshake
- DNS resolution, how a URL becomes a webpage
- HTTP/HTTPS: methods, status codes, headers, statelessness, cookies vs tokens
- REST principles; a first look at GraphQL and WebSockets (real-time)

PHASE 4: Databases (~2-3 weeks)

4.1 Relational databases (primary focus, matches PostgreSQL in your project)

- SQL fundamentals: SELECT/JOIN/GROUP BY/subqueries, writing queries fluently by hand
- Schema design, normalization (1NF-3NF) and when to denormalize
- Indexes: how they speed up reads, and their write-side cost
- Transactions, ACID properties, isolation levels (conceptual)
- N+1 query problem and how ORMs can hide/cause it

4.2 NoSQL, at a conceptual level

- Document stores (MongoDB) vs key-value vs column vs graph databases
- When NoSQL is a better fit than relational, and vice versa

PHASE 5: Software Engineering Practice (~2-3 weeks, then continuous)

5.1 Git & collaboration

- Branching strategies, rebase vs merge, resolving conflicts confidently
- Writing good commit messages and PR descriptions

5.2 Testing

- Unit tests vs integration tests vs end-to-end tests — what each catches
- Test-driven mindset: writing a test before/alongside the feature
- Tools relevant to your stack: Jest/Vitest, React Testing Library

5.3 Code quality & design

- SOLID principles and clean code habits (naming, small functions, avoiding duplication)
- Common design patterns: factory, singleton, observer, strategy — recognize them, don't over-apply them
- Code review etiquette, both giving and receiving feedback

5.4 Delivery basics

- What CI/CD is and why it exists (you don't need to master pipelines yet, just the concept)
- Environment variables, basic deployment concepts (build step, hosting, logs)

PHASE 6: System Design Basics (~2-3 weeks)

6.1 Core concepts

- Scalability: vertical vs horizontal scaling
- Load balancing, caching (client, CDN, server, database layers)
- CAP theorem, consistency vs availability trade-offs (conceptual)
- Database read replicas, sharding — high-level understanding
- Message queues and async processing (why decouple services)

6.2 Practice designing

- Start small: design a URL shortener, then a rate limiter, then a chat app
- Practice the framework: clarify requirements -> estimate scale -> high-level design -> deep dive -> trade-offs
- This becomes relevant for mid-level+ interviews; junior interviews focus more on DSA and fundamentals

PHASE 7: Security Fundamentals (lightweight, ongoing) (~ongoing, ~1 week focused)

- Common web vulnerabilities: XSS, CSRF, SQL injection — what they are and how to prevent each
- Authentication vs authorization; sessions, JWTs, OAuth basics
- Password hashing (bcrypt/argon2) vs encryption — know the difference
- HTTPS/TLS at a conceptual level, environment secrets management

PHASE 8: Engineering with AI (AI-era skills) (~ongoing from the start)

8.1 Using AI tools well (not just chatting)

- Effective prompting for code: give context, constraints, and ask for reasoning/tests, not just answers
- Using AI as a tutor to explain unfamiliar code line-by-line, and as a reviewer for your own code
- Knowing when NOT to trust AI output: verify DSA solutions and security-sensitive code yourself

8.2 Building with AI APIs (feeds directly into your TaskForge AI project)

- How LLM APIs work at a basic level: prompts, tokens, context windows, streaming responses
- Prompt engineering fundamentals: system prompts, few-shot examples, structured output (JSON mode)
- Retrieval-Augmented Generation (RAG) basics: embeddings, vector search, when it's needed vs overkill
- Practical integration: calling an LLM API from a Node/Express backend, handling errors and rate limits

This phase is 'AI-era' in the sense of building AI-powered features and using AI as a tool — it is not a machine learning / data science curriculum, which is a much larger separate track.

PHASE 9: Interview Preparation (~start early, run continuously)

9.1 Technical interview skills

- Practice explaining your thought process out loud while solving problems, not just solving silently
- Timed mock problems (45 min) once DSA patterns feel comfortable
- Mock interviews with peers or on platforms designed for it, once a base of ~100-150 problems is built

9.2 Behavioral & portfolio

- Prepare 5-6 STAR-format stories from your projects (situation, task, action, result)
- Be ready to explain TaskForge AI in depth: architecture decisions, trade-offs, what you'd do differently
- A clean GitHub profile and 2-3 polished, well-documented projects beat many half-finished ones

9.3 Closing the experience gap

- Contribute to a small open-source project or fix a real (even minor) issue — genuine collaboration experience
- Write short technical posts/notes on what you learn; explaining a topic solidifies it and builds a visible trail

Suggested phase order at a glance

Phase	Focus	Rough duration
1	JS deep dive + TypeScript + paradigms	3-4 weeks
2	Data structures & algorithms	8-10 weeks (+ ongoing)
3	OS, memory, networking basics	3-4 weeks
4	Databases (SQL focus + NoSQL concepts)	2-3 weeks
5	Git, testing, clean code, CI/CD basics	2-3 weeks
6	System design basics	2-3 weeks
7	Security fundamentals	~1 week, ongoing
8	AI-era engineering skills	ongoing from day 1
9	Interview prep (DSA + behavioral)	start by phase 2, continuous

Phases overlap in practice — DSA (2) and AI-era skills (8) should run in parallel with everything else, and interview prep (9) should start as soon as you have enough DSA under your belt, not after finishing every phase.